

Session 08 - Connecting Aws S3 To Snowflake – External Stage & Copy Command (detailed Guide).

Loading data into Snowflake tables using external stages and COPY commands

Topic Overview (What This Session Is About)

This session introduced the **end-to-end data migration architecture used in real Snowflake projects**. The discussion started from **data sources**, moved through **data lakes (AWS S3)**, and ended with **loading data into Snowflake tables using external stages and COPY commands**. The goal was to help learners understand **why a data lake is needed**, **how cloud storage fits into Snowflake**, and **how data actually reaches Snowflake for reporting and analytics**.

1. End-to-End Data Migration Flow to Snowflake

Explanation

In real-world projects, Snowflake does **not directly connect to source databases**. Instead, data is first **extracted as files**, stored in a **data lake**, and then loaded into Snowflake.

Typical Flow

1. **Source Systems**
 - SQL Server, Oracle, MySQL
 - Applications, APIs, IoT devices
2. **Extraction**
 - Data extracted as files (CSV, JSON, Parquet)
3. **Data Lake**
 - Raw files stored without transformation
4. **Snowflake**
 - Data loaded into tables
5. **Reporting Tools**
 - Power BI, Tableau

Why This Approach?

- Decouples source systems from Snowflake
 - Supports large volumes of data
 - Easy reprocessing and recovery
-

2. What is a Data Lake?

Definition

A **Data Lake** is a centralized storage repository that holds **all organizational data in its raw format**, without applying transformations at the time of storage.

Key Characteristics

- Stores raw data
 - Supports all data types
 - Schema applied later (schema-on-read)
 - Cheap and scalable storage
-

3. Types of Data Explained

Structured Data

- Fixed schema
- Stored in rows and columns
- Examples:
 - CSV
 - TSV
 - Relational tables

Semi-Structured Data

- Flexible schema
- Key-value or hierarchical format
- Examples:
 - JSON
 - XML
 - Parquet
 - ORC

Unstructured Data

- No predefined structure
- Requires processing or AI tools

- Examples:
 - Audio recordings
 - Videos
 - Images
 - PDFs

Example: Customer call recordings analyzed for sentiment

4. Data Lake vs Data Warehouse

Detailed Comparison

Data Lake	Data Warehouse
Raw data	Transformed data
All data types	Only structured data
Schema-on-read	Schema-on-write
Used for ML, AI	Used for BI reporting
Low-cost storage	Optimized query performance

5. Cloud Platforms for Data Lakes

Storage Services

- **AWS** → S3 (Simple Storage Service)
- **Azure** → Blob Storage / ADLS
- **GCP** → Google Cloud Storage

Snowflake Advantage

Snowflake can connect to **all three clouds** using external stages.

6. AWS Free Trial Account Creation

Why AWS Account Is Needed

- To create a data lake
- To practice real Snowflake projects

Important Points

- Use a new email ID
- Free for 6 months or \$200 credits
- Account closes automatically if not upgraded

Payment Verification

- ₹2 refundable charge
 - UPI / Debit / Credit card
-

7. AWS Login & Security

Root User Login

- Uses registered email
- Full access to AWS account

MFA (Multi-Factor Authentication)

- Adds extra security
 - Uses Google Authenticator
 - Generates 6-digit OTP every 30 seconds
-

8. AWS S3 – Storage Service

What is S3?

- Object-based storage
- Highly scalable
- Used as data lake storage

Why S3 for Snowflake?

- Cheap
 - Durable
 - Supports huge data volumes
-

9. S3 Bucket Creation Rules

Bucket Concept

- Bucket = top-level container
- Stores all folders and files

Naming Rules

- Must be globally unique
- No duplicate names allowed

Best Practice

companyname-raw-bucket

Example: mg-stanley-raw-bucket

Region Selection

- Choose nearest region to client
- Example: Asia Pacific (Mumbai)

10. Data Lake Folder Structure (Industry Standard)

Structure

Bucket

```
└ Database_Name
  └ Schema_Name
    └ Table_Name
      └ Load_Date
```

Example

raw-bucket

```
└ DW
  └ LOAN_SCHEMA
    └ CUSTOMER
      └ 2025-12-10
```

Benefits

- Easy debugging
- Scalable
- Clean separation of data

11. Role of ETL Tools

In Real Projects

- Files are not uploaded manually
- ETL tools extract data automatically

Common Tools

- Informatica
 - Talend
 - Azure Data Factory
-

12. External Stage in Snowflake

Definition

An **external stage** is a Snowflake object used to **connect to external cloud storage**.

Purpose

- Snowflake cannot read S3 directly
 - Stage provides authentication & path
-

13. Authentication Methods for External Stage

Storage Integration (Production)

- Most secure
- Uses IAM role

Access Key & Secret Key (Training)

- Acts like username/password
 - Easy to configure
-

14. IAM Access Key & Secret Key

Steps

1. Go to IAM service
2. Create or select user
3. Generate access key
4. Download CSV

Secret key visible only once

15. Creating External Stage in Snowflake

Required Inputs

- Stage name
- S3 URL
- Access key
- Secret key
- Directory enabled

S3 URL Format

S3://bucket-name/folder-path/

16. Listing Files from Stage

LIST @customer_ext_stage;

Shows all files available for loading

17. File Format Creation

Why File Format?

- Tells Snowflake how to read file

CREATE OR REPLACE FILE FORMAT csv_format

TYPE = CSV

SKIP_HEADER = 1;

18. Loading Data Using COPY INTO

Step 1: Clear Table

TRUNCATE TABLE CUSTOMER;

Step 2: Load Data

COPY INTO CUSTOMER

FROM @customer_ext_stage

FILE_FORMAT = csv_format;

Result

- 20 rows loaded successfully

Key Takeaways

- Data lake is mandatory in Snowflake projects
- AWS S3 is commonly used
- External stage connects Snowflake to cloud
- COPY INTO is core loading command
- Same logic works across AWS, Azure, GCP

What's Next

- Advanced COPY command options
- Error handling
- JSON & Parquet loading
- Automation using tasks

Connecting Aws S3 To Snowflake – External Stage & Copy Command (detailed Guide)

Why Snowflake Needs External Stages

Important Concept

Snowflake **cannot directly read files from AWS S3**.

Reason

- Snowflake is a **data warehouse**, not a storage system
- AWS S3 is **external cloud storage**
- A secure connection is required

Solution

👉 External Stage

An **external stage** acts as a **bridge/connector** between Snowflake and cloud storage.

End-to-End Connection Flow

1. Data exists in AWS S3 (CSV/JSON/Parquet)
 2. Snowflake creates an **External Stage** pointing to S3
 3. Snowflake authenticates using AWS credentials
 4. Files are accessed through the stage
 5. COPY INTO command loads data into Snowflake tables
-

Step 1: Prepare Data in AWS S3

Navigation (AWS Console)

1. Login to **AWS Console**
2. Go to **Services** → **S3**
3. Open your **bucket**
4. Navigate folder path:

Bucket → Database → Schema → Table → Date

Purpose

- Acts as **Data Lake**
- Stores raw data files
- Allows reprocessing if needed

Step 2: Create Target Table in Snowflake

Navigation (Snowflake UI)

1. Login to **Snowflake Web UI**
2. Select:
 - Warehouse
 - Database
 - Schema
3. Go to **Worksheets**

Purpose

- Defines structure where data will be loaded
- COPY command needs a target table

Example

```
CREATE OR REPLACE TABLE CUSTOMER (  
  ID INT,  
  FIRST_NAME STRING,  
  LAST_NAME STRING  
);
```

Step 3: Create File Format (How to Read File)

Why File Format is Required

Snowflake must know:

- File type (CSV, JSON)
- Header rows
- Delimiters

Navigation

- Worksheets → Run SQL

Example (CSV)

```
CREATE OR REPLACE FILE FORMAT csv_ff  
  
TYPE = CSV  
  
SKIP_HEADER = 1  
  
FIELD_DELIMITER = ',';
```

Purpose

- Standardizes file reading
 - Reusable across COPY commands
-

Step 4: Create IAM Access Key (AWS Authentication)

Navigation (AWS Console)

1. Services → **IAM**
2. Users → Select User
3. Security credentials
4. Create **Access Key**
5. Download CSV

Purpose

- Acts like **username & password**
- Allows Snowflake to access private S3 bucket

⚠ Secret key is shown only once

Step 5: Create External Stage in Snowflake

Navigation (UI Method)

1. Snowflake UI
2. Database → Schema
3. Click **Stages**
4. Click **Create → External Stage**

Required Inputs

- Stage Name
- URL:

S3://bucket-name/folder-path/

- AWS Access Key
- AWS Secret Key
- Enable **Directory Table**

Purpose

- Stores connection details
 - Provides reusable reference to S3
-

External Stage – SQL Equivalent

```
CREATE OR REPLACE STAGE customer_ext_stage
```

```
URL = 's3://my-raw-bucket/DW/LOAN_SCHEMA/CUSTOMER/'
```

```
CREDENTIALS = (
```

```
  AWS_KEY_ID = 'XXXX'
```

```
  AWS_SECRET_KEY = 'YYYY'
```

```
)
```

```
DIRECTORY = (ENABLE = TRUE);
```

Step 6: Validate Connection – LIST Command

Why LIST is Important

- Confirms Snowflake can access S3
- Prevents COPY failures

Command

```
LIST @customer_ext_stage;
```

Output

- Shows all files in S3 path
-

Step 7: Truncate Target Table (Optional)

Purpose

- Clean load
- Avoid duplicate data

```
TRUNCATE TABLE CUSTOMER;
```

Step 8: Load Data Using COPY INTO

COPY Command Purpose

- Loads data from stage → table
- Supports parallel processing

Basic Syntax

COPY INTO CUSTOMER

FROM @customer_ext_stage

FILE_FORMAT = csv_ff;

What Happens Internally

- Snowflake reads files from S3
 - Parses data using file format
 - Inserts rows into table
-

Step 9: Verify Loaded Data

SELECT * FROM CUSTOMER;

Purpose

- Confirm successful load
 - Validate row count
-

Why External Stage is Mandatory (Interview Point)

- Security isolation
 - Reusability
 - Centralized connection management
 - Required for automation
-

Production vs Training Approach

Training	Production
Access key	Storage integration
Manual COPY	Automated pipelines
UI-based	IaC / SQL-based

Common Mistakes to Avoid

- Wrong S3 URL format
- Missing trailing slash in S3 path
- Column order mismatch
- File format mismatch
- Not validating with LIST

Real Project Loading Flow

1. ETL places file in S3
2. Snowflake task triggers
3. COPY command executes
4. Errors logged
5. Reporting refreshed

Key Takeaways

- External stage = connection bridge
- File format = file reader
- COPY INTO = data loader
- LIST = connection validator
- Same logic across AWS, Azure, GCP

What Comes Next

- Error handling options
- JSON & Parquet loading
- Validation mode
- Automation using Tasks

Here are **10 most-asked interview questions** on
“**Loading data into Snowflake using External Stages and COPY command**”
(asked in **real Snowflake interviews**, from 2–8 years experience).

1 What is an External Stage in Snowflake? Why do we need it?

Answer:

An **External Stage** is a Snowflake object used to connect Snowflake to **external cloud storage** such as **AWS S3, Azure Blob, or GCS**.

Snowflake cannot directly read external files, so an external stage acts as a **secure bridge** that stores the cloud path and authentication details. It enables Snowflake to access files for loading data using the COPY command.

2 What are the different ways to authenticate Snowflake with AWS S3?

Answer:

There are **two authentication methods**:

1. **Storage Integration (Preferred / Production)**
 - Uses IAM role
 - More secure
 - Used in real projects
 2. **Access Key & Secret Key (Training / POC)**
 - Acts like username/password
 - Easy to configure
 - Not recommended for production
-

3 What is the role of a File Format in Snowflake data loading?

Answer:

A **File Format** tells Snowflake **how to read the data file**.

It defines:

- File type (CSV, JSON, Parquet)
- Header rows
- Field delimiter
- Enclosure character
- Null handling

Without a file format, Snowflake cannot correctly parse external files.

4 Explain the complete flow of loading data from AWS S3 into Snowflake.

Answer:

1. Data is extracted as files and placed in **AWS S3**
 2. Snowflake creates an **External Stage** pointing to S3
 3. A **File Format** is defined
 4. The **COPY INTO command** loads data from stage to table
 5. Data is validated and used for reporting
-

5 What is the purpose of the LIST command in Snowflake?

Answer:

The LIST command is used to **verify connectivity** between Snowflake and the external stage. It shows all files available in the S3 path and confirms that authentication and URL configuration are correct before running the COPY command.

6 What happens internally when a COPY INTO command is executed?

Answer:

Internally, Snowflake:

- Reads files from the external stage
 - Parses data using the file format
 - Loads data in **parallel**
 - Tracks file metadata to avoid duplicate loads
 - Returns load status (success/failure)
-

7 What is the difference between Internal Stage and External Stage?

Answer:

Internal Stage	External Stage
Stored inside Snowflake	Stored in cloud storage
Used for small/intermediate data	Used for large-scale data
No external authentication	Requires cloud authentication
Limited storage	Highly scalable

8 Why is a trailing slash (/) important in S3 stage URL?

Answer:

The trailing slash ensures Snowflake treats the path as a **directory** rather than a file. Without it, Snowflake may fail to locate files correctly or load only one object.

9 Can Snowflake load data from Azure or GCP using the same COPY command?

Answer:

Yes.

The **COPY command syntax remains the same** across AWS, Azure, and GCP.

Only the **authentication method changes**:

- AWS → Access key / Storage integration
 - Azure → SAS token
 - GCP → Service account key
-

10 Why is External Stage preferred over direct file upload?

Answer:

External stages are preferred because they:

- Support large-scale data loads
 - Are reusable and secure
 - Enable automation
 - Match real production architecture
 - Decouple Snowflake from source systems
-

★ Bonus Tip (Interview Gold Line)

“In real projects, Snowflake never loads data directly from source systems. All data first lands in a data lake, and external stages are used to securely load it into Snowflake.”